Réponse TP Cassandra

Question 1 - Expliquez brièvement ce que signifie UN et le rôle de dc1 et rack1

- `U` = Up
- `N` = Normal
- `UN` **= nœud actif et opérationnel**

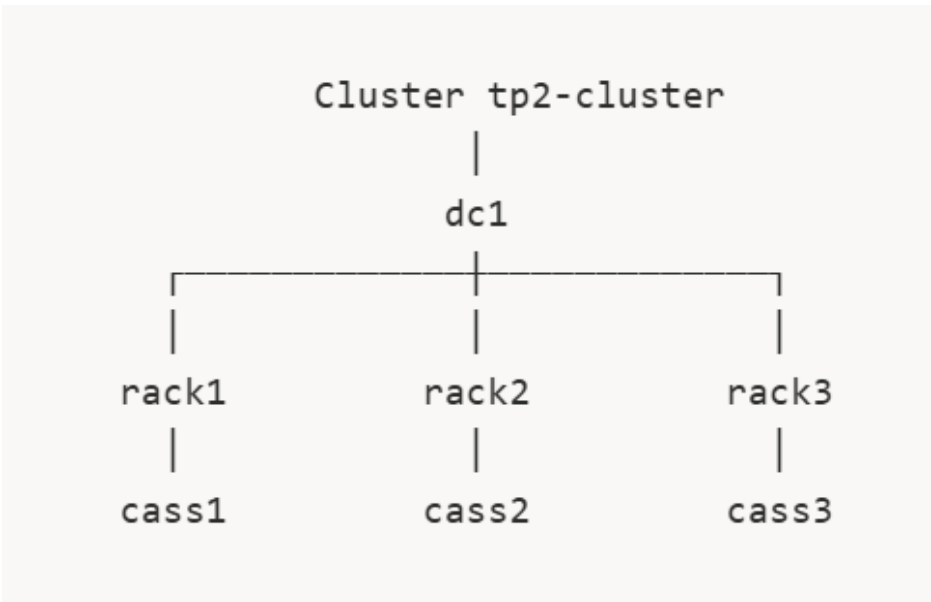
dc1 est le datacenter qui englobe tout, rack1 peut être imaginé comme une baie de serveurs avec la machine cass1 à l'intérieur

Question 2 - Expliquez la différence entre les états UN et UJ

- **UN (Up, Normal)** : le nœud fait pleinement partie du cluster. Il a ses plages de tokens, stocke des données et répond aux requêtes.
- **UJ (Up, Joining)** : le nœud est démarré et il est en train de rejoindre le cluster.

UJ signifie qu'il n'est pas encore pleinement opérationnel, il passera ensuite en UN.

Question 3 - Présentez l'architecture :



On a dc1 qui est le datacenter, dedans on a trois racks, chaque rack a une machine cassandre opérationnelle.

Question 4 - Expliquez simplement :

```
NetworkTopologyStrategy
dc1
RF = 3

Dans un cluster de trois nœuds, que signifie RF=3 pour une partition ?
```

NetworkTopologyStrategy est la stratégie de réplication : la règle qui décide où Cassandra place les copies.

dc1 est le datacenter qui englobe tout le projet. RF3 indique que 3 copies seront envoyées dans les noeuds. Si on a 6 noeuds, le noeud coordinateur en choisira 3 où dupliquer les données

Question 5 -

Pourquoi ville_id comme clé de partition ?

Toutes mes requêtes demandent des relevés pour une ville donnée. Avec ville_id comme clé de partition, tous les relevés d'une ville sont stockés ensemble, sur les mêmes réplicas. Une requête n'a donc qu'une seule partition à lire, sans solliciter tout le cluster. Dans la table villes, ville_id est aussi la clé, puisqu'on cherche une ville par son identifiant.

Pourquoi une clé de clustering (observed_at) ?

Une ville a plusieurs relevés au fil du temps, il faut donc un second élément pour distinguer chaque ligne dans la partition. observed_at joue ce rôle, et il trie en plus les lignes par heure sur le disque. C'est ce qui rend efficaces les requêtes de type « dernier relevé » (ORDER BY … DESC LIMIT 1), « plage horaire » (>= et <=) et « heure précise » (=). La table villes n'en a pas besoin : une ville = une ligne.

```
cqlsh:meteo> SELECT temperature_2m
         ... FROM releves_temperature
         ... WHERE ville_id = 'paris'
         ...   AND observed_at = '2026-10-06 15:00:00';

 temperature_2m
----------------
           24.2
```

Comment le modèle répond aux requêtes métier ?

Chaque requête REQ-01 à REQ-05 fournit la clé de partition (ville_id = 'paris') et, quand c'est utile, filtre sur la clustering key. Cassandra sait donc directement où chercher, ce qui explique qu'aucune requête n'a besoin d'ALLOW FILTERING. Les coordonnées sont dans une table séparée (villes) pour ne pas les répéter dans chaque relevé.

```
cqlsh:meteo> SELECT nom, latitude, longitude
         ... FROM villes
         ... WHERE ville_id = 'paris';

 nom   | latitude | longitude
-------+----------+-----------
 Paris |    48.85 |      2.35
```

Limite à connaître : ce modèle ne permet pas de chercher « les heures où la température dépasse 20 °C » sans ALLOW FILTERING, car temperature_2m n'est pas une clé. Si ce besoin existait, il faudrait créer une autre table pensée pour cette requête, plutôt que d'ajouter ALLOW FILTERING. Il ne fait pas partie de tes 5 besoins métier.